

VERSION 2.4.2 · MIT · 100% LOKAL · ZERO TELEMETRY



Text Watermark Studio

Das lokale Forensik-Labor für KI-Text- und Datei-Wasserzeichen

*KI-Texte erkennen. Wasserzeichen setzen. Beweise liefern,
die man verteidigen kann — statt „sieht aus wie KI“-Gefühlen.*

1.113
TESTS

15+
CLI-COMMANDS

14×
SPEEDUP

8
PLUGINS

3
SIGNATUREN

github.com/mrmixx-max/text-watermark-studio · MIT License

`pip install text-watermark-studio`

Alles, was Sie für KI-Wasserzeichen-Forensik brauchen

Text Watermark Studio deckt den gesamten Workflow ab — von der Detektion über die Beweisführung bis zur aktiven Markierung. Kein Cloud-Dienst, kein Telemetry, keine Abhängigkeiten.

**KGW-Detektion mit Z-Score**

Statistische Detektion von Greenlist-Wasserzeichen. $Z \geq 4,0$ = watermark_detected. Multi-Key mit Bonferroni-Korrektur.

**Multi-Bit Payload Watermarking**

Invariant-Feature Codebuch (Yoo et al., ACL 2023). Embed/Extract von User-ID, Timestamps, Run-IDs.

**HMAC + ML-DSA Signierung**

Forensik-Reports signieren (HMAC-SHA256) oder quantenresistent (ML-DSA FIPS 204). Verifizierbar, auditierbar.

 **ΔZ -Messung & Beweisführung**

Messen Sie die Wasserzeichen-Stärke vor und nach einem Angriff. Mit Quittung — nicht nur Behauptung.

**Remove & Rewrite Engine**

Best-effort Wasserzeichen-Entfernung: clean → dilute → structural rewrite. Ehrlich, was sich ändert.

**Findings-Befund (C5)**

KI-Erklärungs-Befund mit Evidenzklassen A–D, Prüfpriorität 0–5, signiert. Für Gerichte und Gutachten.

**Web Dashboard (HTMX)**

5 Seiten: Overview, Watermark, Files, Reports, Settings. SSE-Echtzeit-Statistiken. Kein Build-Step.

**Multi-Model LLM Router**

Fallback, Round-Robin Load Balancing, Health Checks. Automatisch bei Ollama-Modellwechsel.

**Prometheus + Grafana**

8 Metriken, Alerting-Regeln, Docker Compose Stack. Production Monitoring out-of-the-box.

**Timing-Safe Security**

hmac.compare_digest, JWT Auth, Rate Limiting (100/min), API Key Rotation (5 Keys), SSRF-Prävention.

**Batch & Watch Mode**

Batch-Embed mit --verify ($Z > 4$ -Bestätigung). Verzeichnis-Überwachung mit KGW-Text-Detektion.

**--verify Flag**

File-Clean inspiziert die bereinigten Bytes erneut. Beweist, dass der Strip funktioniert hat.

**--quiet Flag**

Maschinenlesbare Ausgabe für Scripting/Pipelines. Status-Meldungen nur auf stderr.

**Code/Audio/Video Plugins**

KI-generierten Code erkennen, SynthID-Style Audio-Metadaten, C2PA/XMP in Video-Containern.

Drei Ebenen. Drei Signaturen. Eine Antwort.

Detektions-Ebenen

Ebene 1 — Unsichtbare Zeichen

- ✓ Bidirektionale Steuerzeichen (LRE, RLE, LRO, RLO)
- ✓ Zero-Width-Spaces, Joiner, Tag-Blöcke
- ✓ Formatzeichen, Unicode-Steganographie

Ebene 2 — Stil-Marker

- ✓ 18 KI-Phrasierungsmuster + gebeugte Formen
- ✓ Statistische Fingerabdrücke in der Wortwahl

Ebene 3 — Statistische Wasserzeichen

- ✓ KGW Greenlist (Kirchenbauer et al.)
- ✓ E-Prozess (anytime-valid, Early-Stopping)
- ✓ Signature-Filter (FPR-Kontrolle, arXiv 2606.18430)

KGW-Detektion

```
ai-wm detect artikel.txt --key demo-kgw-1
# Z = 15,9 → watermark_detected

ai-wm detect artikel.txt --key falscher-key
# Z = -0,2 → no_signal

ai-wm delta-z ORIGINAL.txt --transform clean --key K
# ΔZ: 15,9 → 0,4 → Wasserzeichen degradiert
```

Signaturalgorithmen

HMAC-SHA256 (Standard)

Zero deps. stdlib-only. Für interne Auditierung.

ML-DSA-44 / 65 / 87 (FIPS 204)

Quantenresistent. Für Gutachten, die 2030+ gültig bleiben müssen. cryptography≥50.

Sicherheits-Hardening

- ✓ Timing-safe API Key Vergleich (hmac.compare_digest)
- ✓ SSRF-Prävention via URL-Schema-Validierung
- ✓ Rate Limiting (100 req/min pro API-Key)
- ✓ JWT Token Auth mit Scopes
- ✓ API Key Rotation (5 aktive Keys, Expiry)
- ✓ Request Logging Middleware

Datei-Formate

```
ai-wm file-inspect bericht.pdf
# {found, valid, key_id, algorithm, provenance}

ai-wm file-clean foto.jpg -o sauber.jpg --verify
# C2PA, EXIF, XMP – verified_clear | residual_hard_bound

ai-wm file-embed d.pdf --key K -o signiert.pdf
# HMAC-Provenance-Marke in 8 Formaten
```

Jeder Befehl druckt jetzt sein Ergebnis

Früher endeten viele Handler mit `else: pass`. Jetzt druckt jeder Befehl JSON oder `--pretty`. Exit-Codes: 0 sauber, 1 Funde, 2 Fehler.

Kern-Kommandos

```
ai-wm detect [file] --key K --level word|bpe # KGW + Unicode  
+ Phrasierung ai-wm clean [file] --nfkc --fold-confusables #  
Unicode-Layer entfernen ai-wm dilute [file] --intensity  
light|standard|aggressive # Phrasen umschreiben (33 Regeln)  
ai-wm embed [file] --key K --gamma 0.25 # Greenlist-Marke  
setzen ai-wm pipeline [file] --rewrite-mode #  
detect→clean→dilute→rewrite→detect ai-wm remove [file] --use-  
llm --aggressive # Best-effort Entfernung
```

Neue Kommandos (v2.4.2)

```
ai-wm delta-z FILE --transform clean --key K # ΔZ-Messung mit  
Quittung ai-wm finding FILE --frs --sign SECRET # KI-Befund  
(C5), signiert ai-wm trace FILE --window 500 --step 250 # Z-  
Score-Trajectory ai-wm payload embed FILE --payload "user-42"  
# Multi-bit Watermarking ai-wm evade FILE --target-z 3.9 #  
Adversarial Evaluation
```

Datei-Befehle

```
ai-wm file-inspect FILE # C2PA/EXIF/XMP erkennen ai-wm file-  
clean FILE --verify # Bereinigen + Re-Inspekt ai-wm file-  
embed FILE --key K # HMAC-Provenance setzen ai-wm file-detect  
FILE --key K # Provenance verifizieren ai-wm report FILE --  
key K --pdf # HTML/PDF Forensik-Report
```

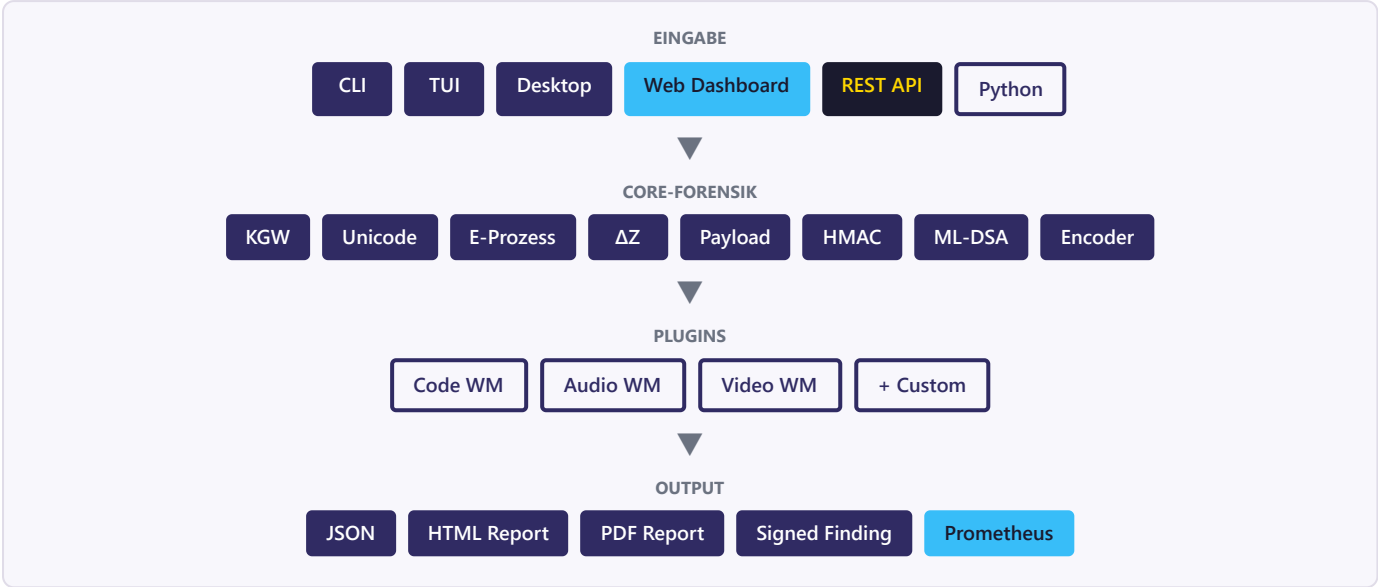
Report-Signierung

```
ai-wm report-keygen --algorithm mldsa-44 # ML-DSA Key-Pair  
generieren ai-wm report-sign FINDING.json --secret S # HMAC-  
SHA256 oder ML-DSA ai-wm report-verify SIGNED.json # Signatur  
prüfen (0/1/2)
```

Servicier-Befehle

```
ai-wm serve --port 8080 # FastAPI Server (22 Module) ai-wm  
dashboard --port 8080 # Web Dashboard (HTMX) ai-wm tui #  
Terminal UI (25 Aktionen) ai-wm watch DIR --kgw # Verzeichnis  
überwachen ai-wm batch IN OUT --verify # Batch mit Z>4  
Bestätigung
```

Plugins, Monitoring, Oberflächen — Eine Codebasis



Monitoring-Stack

```
docker compose -f docker-compose.monitoring.yml up -d
# Prometheus :9090 · Grafana :3000 · Alerting

Metriken: http_requests_total · watermark_operations_total · detection_score_histogram · verdict_total · stream_pending ·
dlq_depth · dlq_replays_total
```

API-Module (22 Routen)

Forensics · Metadata · Documents · PDF · RAG · LLM · Routing · Prompts · Optimization · Multi-Agent · Graph · Community · Rewrite · Export · Cloud-Upload · Queue/Streams · Auth · Health · Metrics

Wir sagen, was wir nicht können.

Jede Technologie hat Grenzen. Wir dokumentieren die ehrlich, statt sie zu verstecken. Das ist der Unterschied zwischen einem Forensik-Labor und einem "KI-Detektor", der nur Style-Matching macht.

Was wir können

- ✓ Statistische Wasserzeichen mit Z-Score messen (reproduzierbar, auditierbar)
- ✓ Eigene Wasserzeichen setzen und verifizieren
- ✓ Datei-Metadaten bereinigen und beweisen, dass es funktioniert hat
- ✓ Reports signieren (HMAC oder ML-DSA) und verifizieren
- ✓ Encoding-Angriffe erkennen (overlong UTF-8, Homoglyphen, versteckte BOMs)

Was wir nicht können (und ehrlich sagen)

- ✓ Jeden KI-Text erkennen — nur mit der passenden Methode und dem passenden Key
- ✓ Statistische Wasserzeichen entfernen — nur umschreiben (Signal degradieren)
- ✓ SynthID-Pixel ohne das 220MB Research-Checkpoint detektieren
- ✓ Dateien ohne Metadaten-Kontext beweisen — nur mit eigener Registry

Vergleich: Wir vs. "KI-Detektoren"

Eigenschaft	TWS	Typ. "KI-Detektor"
Lokale Ausführung	✓ Ja	✗ Cloud
Reproduzierbarer Score	✓ Z-Score	✗ Blackbox
Eigene Wasserzeichen	✓ KGW	✗ Nein
Signierte Beweise	✓ ML-DSA	✗ Nein
Ehrliche Grenzen	✓ Ja	✗ Versprechen
Open Source (MIT)	✓ Ja	✗ Proprietär

Installation in 30 Sekunden

Python 3.10+. Keine Cloud. Kein API-Key. Kein Telemetry.

```
pip install text-watermark-studio
```

Quellcode: github.com/mrmixx-max/text-watermark-studio
MIT License · Dokumentation in docs/ · Issues willkommen